

Computer Vision

Lecture 05: 객체 감지용 커스텀 데이터셋 및 모델 학습 파이프라인 구축

Jaesung Lee

curseor@cau.ac.kr

2025. 04. 02.



Contents

1. 데이터셋 다운로드
2. 논문 탐색
3. 데이터셋, 데이터로더 클래스 정의
4. 모델 forward pass 및 loss 계산

1. 데이터셋 다운로드

- 깃허브 리파지토리
 - https://github.com/tkdgur658/CAU_25_Spring_ComputerVision/tree/main/Week05
- 가상환경 필요 라이브러리 설치 (README05.md 참조)

1. 가상환경 생성/활성화

다음 명령어를 통해 Python 3.10을 사용하는 `ComputerVision` 이라는 이름의 가상환경을 생성합니다.

```
conda create -n ComputerVision python=3.10
```



2. 가상환경 활성화

생성한 환경을 활성화합니다.

```
conda activate ComputerVision
```



3. Jupyter Kernel 등록

Jupyter Notebook에서 가상환경을 사용할 수 있도록 커널(Kernel)을 등록합니다.

```
pip install ipykernel  
python -m ipykernel install --user --name ComputerVision --display-name ComputerVision
```



4. 필요 패키지 설치

강의에서 사용하는 필수 패키지를 설치합니다. (`requirements.txt` 파일이 있는 폴더에서 실행하세요.)

```
pip install -r requirements.txt
```



1. 데이터셋 다운로드

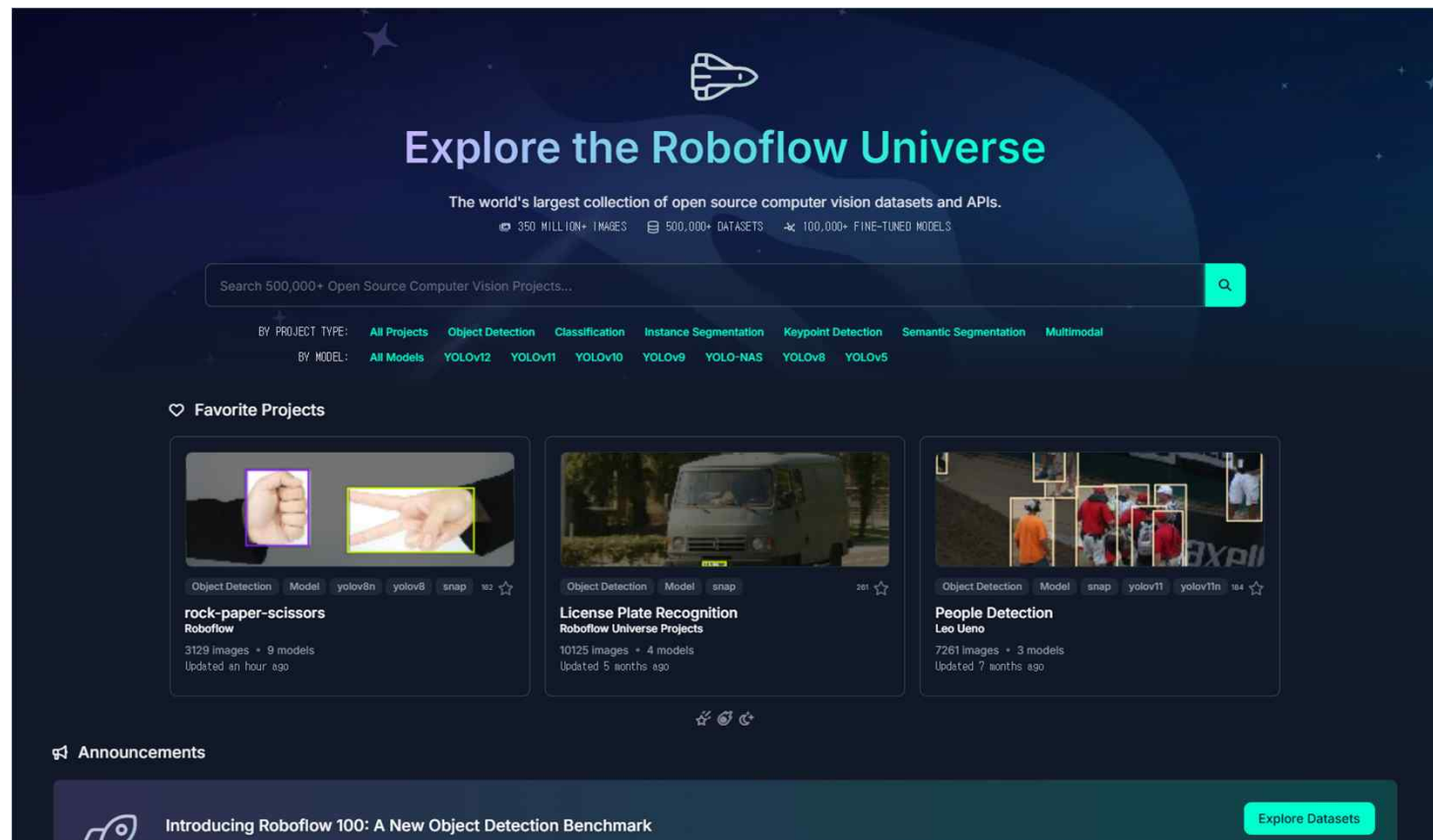
2. 논문 탐색

3. 데이터셋, 데이터로더
클래스 정의

4. 모델 forward pass 및
loss 계산

1. 데이터셋 다운로드

- 실습에 필요한 데이터셋 다운로드를 위한 Roboflow 사이트 접속
 - <https://universe.roboflow.com/>



1. 데이터셋 다운로드

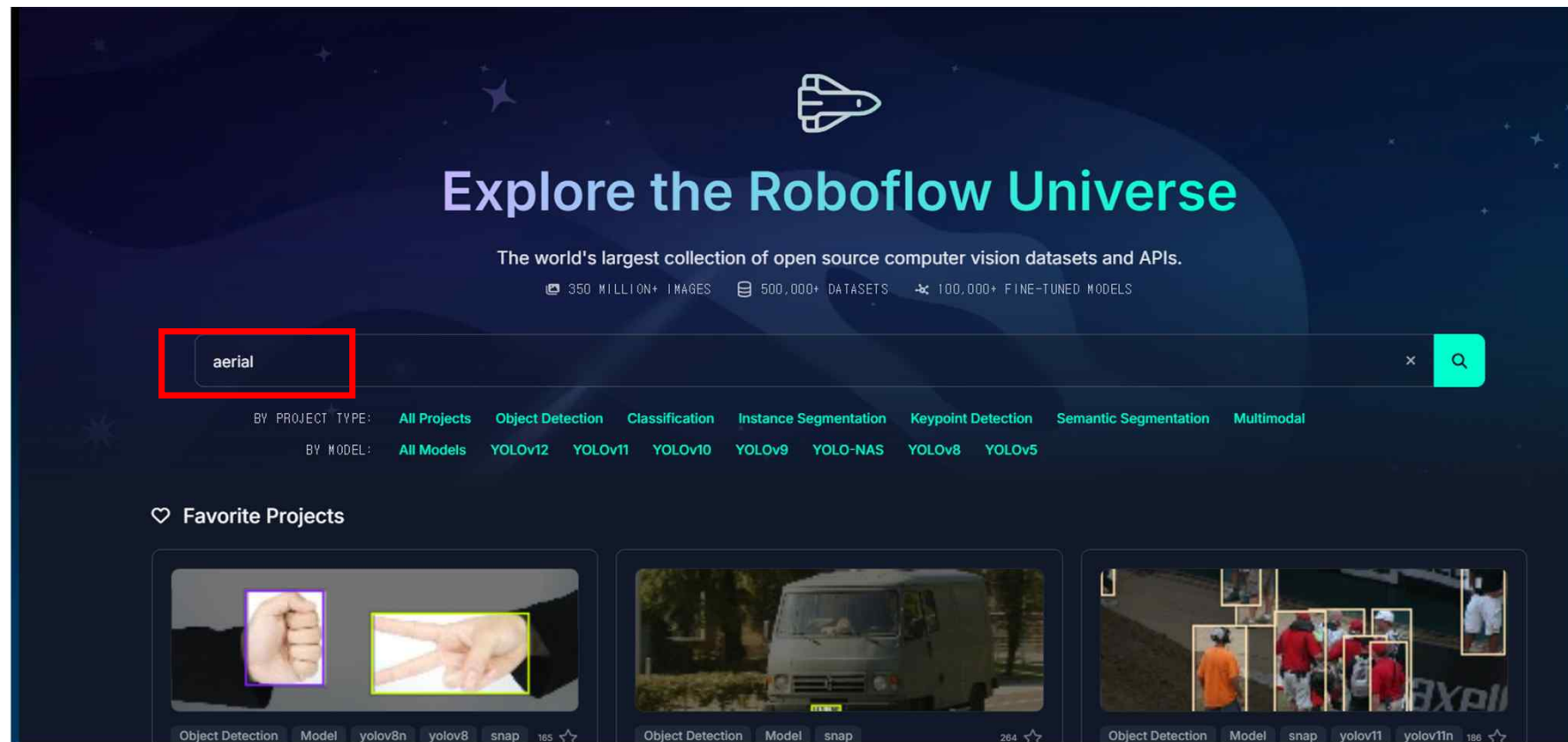
- 검색창에 "Object(원하는 도메인) Detection" 을 입력 후 검색
- 이번 실습에서는 aerial을 입력

1. 데이터셋 다운로드

2. 논문 탐색

3. 데이터셋, 데이터로더
클래스 정의

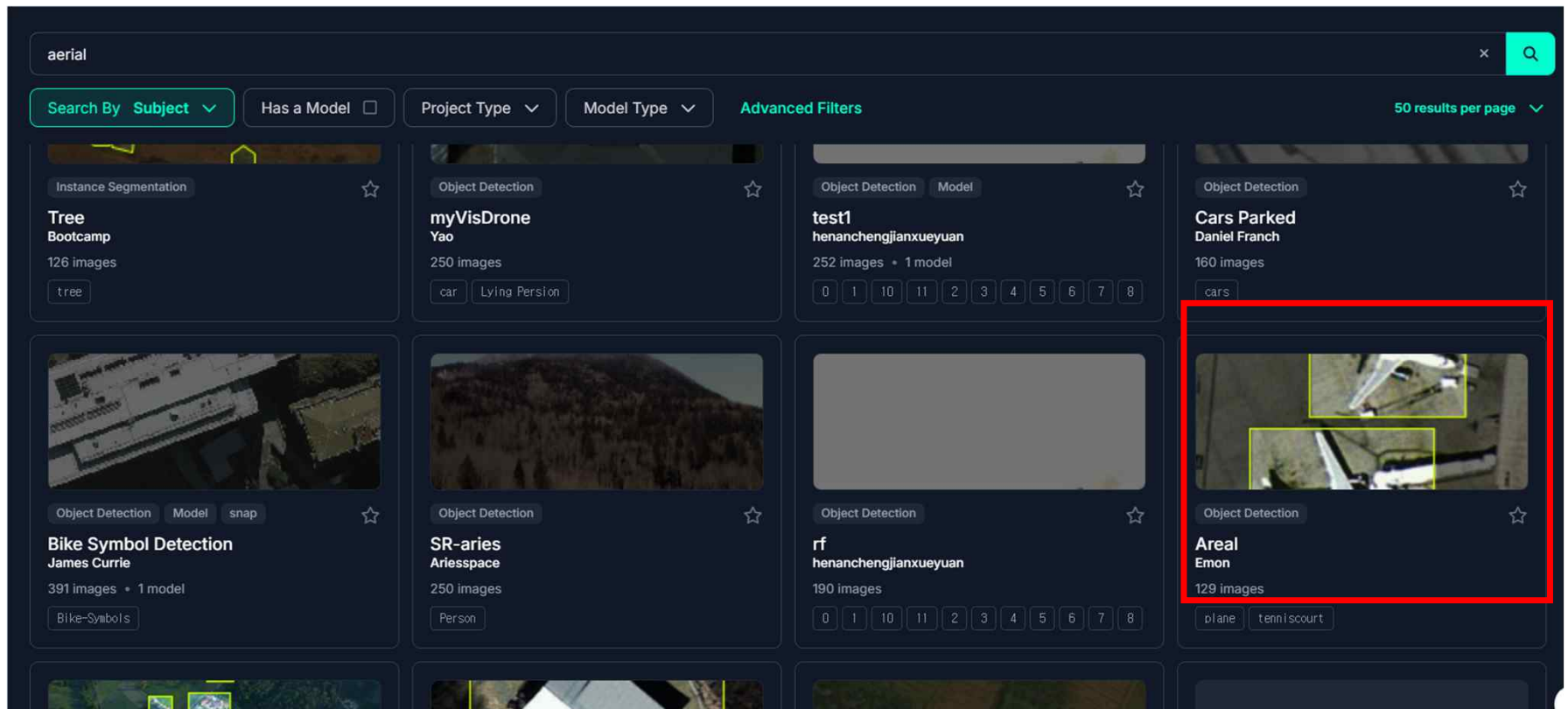
4. 모델 forward pass 및
loss 계산



1. 데이터셋 다운로드

- 여기서는 예시로, "Areal Computer Vision Project" 을 진행

1. 데이터셋 다운로드
2. 논문 탐색
3. 데이터셋, 데이터로더 클래스 정의
4. 모델 forward pass 및 loss 계산



1. 데이터셋 다운로드

- 좌측의 Dataset 을 클릭

The screenshot displays the Areal dataset interface. On the left sidebar, the 'Dataset' tab is highlighted with a red box and a blue indicator. The main content area shows the 'Dataset Versions' section, with the version '2022-06-03 11:10am' selected and highlighted by a green box. Below this, the 'Popular Download Formats' section lists various formats like YOLOv11, YOLOv9, YOLOv8, YOLOv5, YOLOv7, COCO JSON, YOLO Darknet, Pascal VOC XML, TFRecord, PaliGemma, CreateML JSON, and Other Formats. At the bottom, the 'Dataset Split' section shows the distribution: TRAIN SET (72%), VALID SET (19%), and TEST SET (9%).

Dataset Versions

VERSIONS

2022-06-03 11:10am
v1 · 3 years ago

Popular Download Formats

YOLOv11 YOLOv9 YOLOv8 YOLOv5
YOLOv7 COCO JSON YOLO Darknet Pascal VOC XML
TFRecord PaliGemma CreateML JSON Other Formats

129 Total Images [View All Images →](#)

Dataset Split

Dataset Split	TRAIN SET	VALID SET	TEST SET
Percentage	72%	19%	9%

1. 데이터셋 다운로드

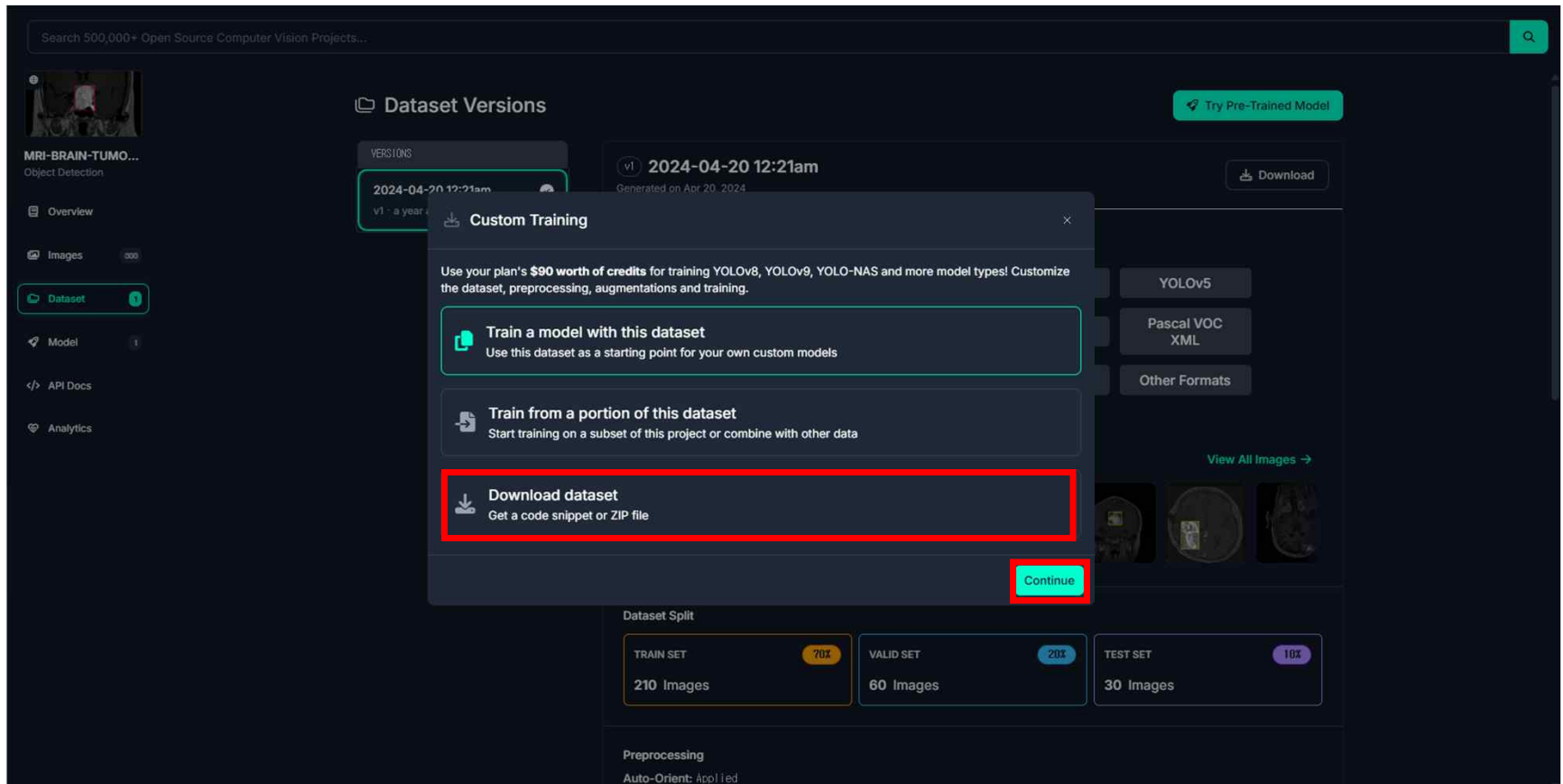
- 이후, 원하는 버전을 선택하고, 데이터 포맷 양식을 선정하여 다운로드

- 1. 데이터셋 다운로드
- 2. 논문 탐색
- 3. 데이터셋, 데이터로더 클래스 정의
- 4. 모델 forward pass 및 loss 계산

The screenshot displays the Areal dataset interface. On the left, a sidebar shows navigation options: Overview, Images (129), Dataset (1), Model (0), API Docs, and Analytics. The main content area is titled 'Dataset Versions'. Under the 'VERSIONS' tab, the version '2022-06-03 11:10am' (v1, 3 years ago) is highlighted. To the right, the 'Popular Download Formats' section lists various formats, with 'YOLOv8' highlighted in a red box. Below this, a section titled '129 Total Images' shows a grid of image thumbnails. At the bottom, the 'Dataset Split' section shows the distribution: TRAIN SET (72%), VALID SET (19%), and TEST SET (9%).

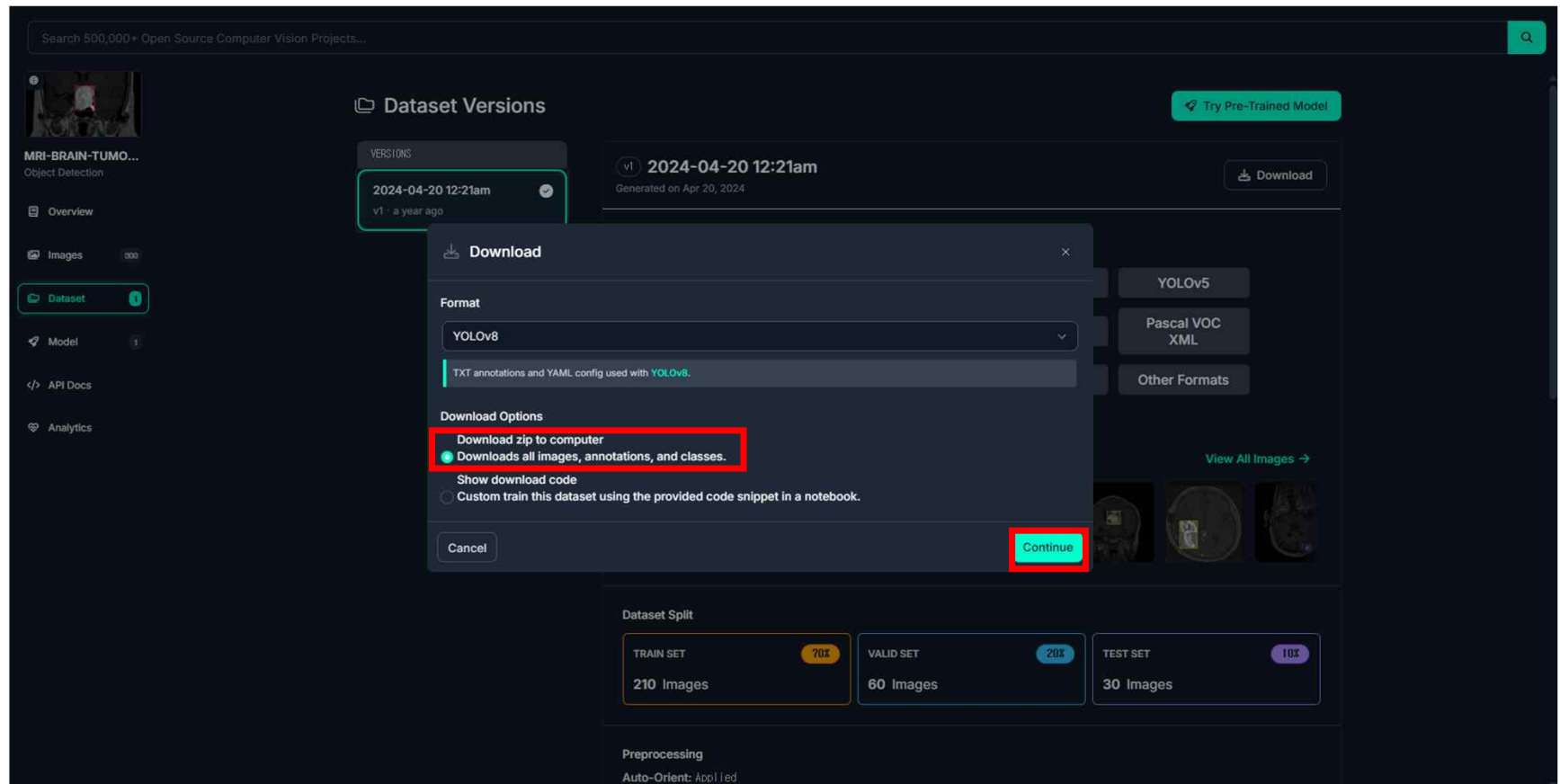
1. 데이터셋 다운로드

- 세 번째의 Download dataset 을 클릭한 후 Continue 버튼을 클릭



1. 데이터셋 다운로드

- 상단의 Download zip to computer 을 클릭한 후 Continue 버튼을 클릭



1. 데이터셋 다운로드 및 모델 로드

- 다운로드 된 zip 파일을 압축 해제 한 후, 내부를 살펴보면 train, test, valid 폴더로 나누어져 있음

📁 / ... / Week05 / Areal.v1i.yolov8 /

Name	Last Modified
test	2 years ago
train	2 years ago
valid	2 years ago
data.yaml	2 years ago
README.dataset.txt	2 years ago
README.roboflow.txt	2 years ago

- 각 폴더 안에는 images와 labels 폴더 안에 각각 이미지 데이터와 텍스트 파일로 구성되어 있음

📁 / ... / test / images /

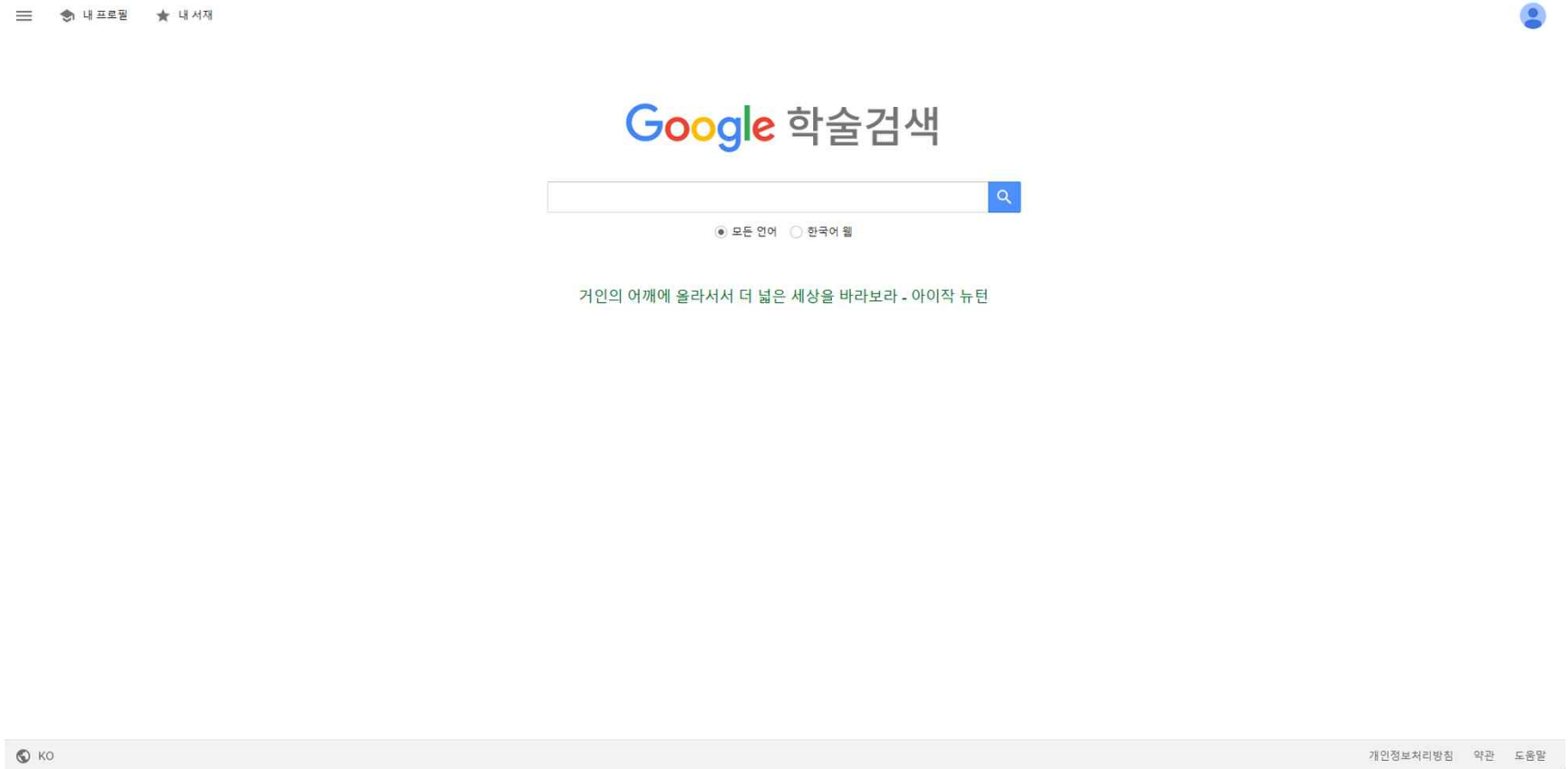
Name	Last Modified
airplane50.jpg.rf.3625b8b8e7827...	2 years ago
airplane52_1.jpg.rf.408255b5eba...	2 years ago
airplane52.jpg.rf.c1683641759e6...	2 years ago
airplane54.jpg.rf.a06263f320148...	2 years ago
airplane58.jpg.rf.896ff89d50377...	2 years ago
airplane60.jpg.rf.5cf79ef7fb56de...	2 years ago
tennis court01_1.jpg.rf.e3a6c960...	2 years ago
tennis court13_1.jpg.rf.6d309675...	2 years ago
tennis court22_1.jpg.rf.2d3738ba...	2 years ago
tennis court23_1.jpg.rf.831b4901...	2 years ago
tennis court27.jpg.rf.19bc653a75...	2 years ago

📁 / ... / test / labels /

Name	Last Modified
airplane50.jpg.rf.3625b8b8e7827...	2 years ago
airplane52_1.jpg.rf.408255b5eba...	2 years ago
airplane52.jpg.rf.c1683641759e6...	2 years ago
airplane54.jpg.rf.a06263f320148...	2 years ago
airplane58.jpg.rf.896ff89d50377...	2 years ago
airplane60.jpg.rf.5cf79ef7fb56de...	2 years ago
tennis court01_1.jpg.rf.e3a6c960...	2 years ago
tennis court13_1.jpg.rf.6d309675...	2 years ago
tennis court22_1.jpg.rf.2d3738ba...	2 years ago
tennis court23_1.jpg.rf.831b4901...	2 years ago
tennis court27.jpg.rf.19bc653a75...	2 years ago

2. 논문 탐색

- Google Scholar 접속 (<https://scholar.google.com/>)



2. 논문 탐색

- 기간 설정 (2021년도 부터)

Google 학술검색

aerial object detection github

Q

학술자료

검색결과 약 17,300개 (0.10초)

모든 날짜

2025년부터

2024년부터

2021년부터

기간 설정...

관련도별 정렬

날짜별 정렬

모든 언어

한국어 웹

모든 유형

검토 자료

☐ 특허 포함

☒ 서지정보 포함

☒ 알림 만들기

Object detection in aerial images: A large-scale benchmark and challenges
[J Ding, N Xue, GS Xia, X Bai, W Yang...](#) - IEEE transactions on ..., 2021 - [ieeexplore.ieee.org](#)
... In this section, we first review **object detection** datasets of natural and **aerial** images. ... **object detectors** in **aerial** images. Finally, we briefly review the code libraries for **object detection**. ...
☆ 저장 ㉞ 인용 476회 인용 관련 학술자료 전체 11개의 버전 Web of Science: 259

Tiny object detection in aerial images
[J Wang, W Yang, H Guo, R Zhang...](#) - 2020 25th international ..., 2021 - [ieeexplore.ieee.org](#)
... tiny **objects** contain a small number of pixels and are easily confused with the background. To advance tiny **object detection** research in **aerial** ... at: <https://github.com/jwwangchn/AI-TOD>. ...
☆ 저장 ㉞ 인용 247회 인용 관련 학술자료 전체 2개의 버전

Recent Advances for Aerial Object Detection: A Survey
[J Leng, Y Ye, M Mo, C Gao, J Gan, B Xiao...](#) - ACM Computing ..., 2024 - [dl.acm.org](#)
... **aerial object detection** ... **aerial object detection** datasets VisDrone-DET and DOTA, and we discuss several future directions that could facilitate the development of **aerial object detection**. ...
☆ 저장 ㉞ 인용 5회 인용 관련 학술자료 Web of Science: 2 ㉞

Oriented reppoints for aerial object detection
[W Li, Y Chen, K Hu, J Zhu](#) - ... of the IEEE/CVF conference on ..., 2022 - [openaccess.thecvf.com](#)
... The source code is availabel at: <https://github.com/...> **objects** with dense-distribution and complex scene in **aerial** images. In this work, we proposed an oriented **object detector** for **aerial** ...
☆ 저장 ㉞ 인용 417회 인용 관련 학술자료 전체 5개의 버전 ㉞

[PDF] [ieee.org](#)
Find it @ CAU

[PDF] [ieee.org](#)
Find it @ CAU

[PDF] [acm.org](#)
Full View

[PDF] [thecvf.com](#)

2. 논문 탐색

- 이 실습에서는 YoloOW 논문을 선정하여 진행

1. 데이터셋 다운로드

2. 논문 탐색

3. 데이터셋, 데이터로더
클래스 정의

4. 모델 forward pass 및
loss 계산

Yoloow: A spatial scale adaptive real-time object detection neural network for open water search and rescue from uav aerial imagery

J Xu, X Fan, H Jian, C Xu, W Bei, Q Ge, T Zhao

IEEE Transactions on Geoscience and Remote Sensing, 2024 • [ieeexplore.ieee.org](#)

Personnel and boat detection in unmanned aerial vehicles (UAVs) imagery plays a crucial role in open water search and rescue missions. The diverse perspectives and altitudes of UAV images often result in significant variations in the imagery's appearance and dimensions of personnel and boats, and the false detections arising from water surface flares are acknowledged as a great challenge as well. The existing deep learning-based detection methods employ convolutional blocks with fixed kernel sizes to extract features

자세히 보기 ▾

☆ 저장 59 인용 16회 인용 관련 학술자료 전체 2개의 버전 Web of Science: 7

[PDF] [ieee.org](#)
Find it @ CAL

YoloOW: A Spatial Scale Adaptive Real-Time Object Detection Neural Network for Open Water Search and Rescue From UAV Aerial Imagery

Jianhao Xu[✉], Xiangtao Fan, Hongdeng Jian, Chen Xu[✉], Weijia Bei[✉], Qifeng Ge, and Teng Zhao[✉]

Abstract—Personnel and boat detection in unmanned aerial vehicles (UAVs) imagery plays a crucial role in open water search and rescue missions. The diverse perspectives and altitudes of UAV images often result in significant variations in the imagery's appearance and dimensions of personnel and boats, and the false detections arising from water surface flares are acknowledged as a great challenge as well. The existing deep learning-based detection methods employ convolutional blocks with fixed kernel sizes to extract features from the imagery at a fixed spatial scale, which will lead to missed and false detections and severely affect detection accuracy when there are substantial differences in the appearance and size of the target objects. In this article, a spatial scale adaptive real-time object detection neural network, namely YoloOW, was proposed to tackle the challenge of personnel and boat detection amid the diverse UAV imagery, which comprises a feature extractor, a feature enhancer, and a postprocessor. The OaohRep convolutional block was proposed as a pivotal component in constructing the YoloOW and applied to the feature extractor and the feature enhancer. Compared with general convolution blocks, the OaohRep convolution block can extract image features across a wide range of spatial scales, show better scale adaptability, and achieve faster detection speed due to its unique merged convolution layer design. OaohRepBi-path aggregation network (PAN) was proposed in the feature enhancer, which imitated the architecture of the classic algorithm scale invariant feature transform (SIFT) and was successfully applied to deep learning models, showing better scale adaptability. A novel UAV detection box filter (UDBF) module was proposed in the postprocessor, which can effectively remove false detections caused by water surface flares. Experimental results demonstrate that our YoloOW model achieves 37.18% mean averaged precision (mAP) on the SeaDronesSee dataset, surpassing the baseline by 8.43%. This paper introduces a new perspective on object detection in the leaderboards. The code will be available at <https://github.com/Xjh-UCAS/YoloOW>.

Manuscript received 28 December 2023; revised 29 February 2024 and 6 April 2024; accepted 25 April 2024. Date of publication 1 May 2024.

Index Terms—Object detection, open water search and rescue, unmanned aerial vehicle (UAV) aerial imagery, unmanned aerial vehicle (UAV) metadata, water surface flares.

I. INTRODUCTION

OPEN water search and rescue is the final line of life safety. The crux of open water search and rescue lies in the rapid detection of distressed individuals in the open water and the precise estimation of their relative positions relative to the rescue boat. At present, almost all open water search and rescue activities need human participation, and the search and rescue efficiency is low [1].

Due to the advancements in oblique photogrammetry and remote sensing, unmanned aerial vehicles (UAVs) have emerged as a promising remote sensing platform, offering advantages, such as high mobility, flexible viewing angles, and cost-effectiveness [2], [3]. UAVs are demonstrating their potential for open water search and rescue.

The primary focus of integrating UAVs into open water search and rescue operations is to achieve automated object detection within UAV aerial imagery [4]. Object detection refers to the process of identifying and locating specific objects within an image or video. In the context of this scenario, it involves identifying elements, such as swimmers, floaters, boats, or other pertinent items present in open water scenes captured by UAVs, which are annotated using bounding boxes. Deep learning-based object detection involves establishing the correlation between image features and bounding boxes during the training phase and during the detection phase generating bounding boxes as an output based on input imagery. Object detection based on deep learning architecture has achieved remarkable breakthroughs and has become the mainstream approach in various applications. For typical scenes such as

3. 데이터셋, 데이터로더 클래스 정의

- 데이터셋과 모델 파일을 다음과 같이 프로젝트 경로에 정리

/ ... / CAU_25_Spring_ComputerVision-main / Week05 /

Name	Last Modified
Areal.v1i.yolov8	an hour ago
cfg	an hour ago
data	38 minutes ago
models	an hour ago
utils	an hour ago
01_Custom_Dataset_and_Model_aerial.ipynb	seconds ago
Areal.v1i.yolov8.zip	an hour ago
data_utils.py	3 hours ago
README05.md	3 hours ago
requirements.txt	3 hours ago

/ ... / Week05 / Areal.v1i.yolov8 /

Name	Last Modified
test	2 years ago
train	2 years ago
valid	2 years ago
data.yaml	2 years ago
README.dataset.txt	2 years ago
README.roboflow.txt	2 years ago

1. Check Custom Dataset and Model

```
[1]: 1 import os
      2 base_data_dir = 'Areal.v1i.yolov8'
      3 base_model_dirs = ['models', 'cfg', 'data', 'utils']
      4 print(os.path.exists(base_data_dir))
      5 for dir in base_model_dirs:
      6     print(os.path.exists(dir))

True
True
True
True
True
```

3. 데이터셋, 데이터로더 클래스 정의

- train/images, train/labels, valid/image, valid/labels 경로 확인
- 커스텀 Dataset 클래스 정의

2. Dataset Class, DataLoader Definition

```
[2]: 1 from data_utils import T, YOLOv8Dataset, DataLoader, yolo_collate_fn, visualize_batch
2
3 train_img_dir = os.path.join(base_data_dir, "train", "images")
4 train_label_dir = os.path.join(base_data_dir, "train", "labels")
5 val_img_dir = os.path.join(base_data_dir, "valid", "images")
6 val_label_dir = os.path.join(base_data_dir, "valid", "labels")
7 # Add test paths similarly if needed
8 # test_img_dir = os.path.join(base_data_dir, "test", "images")
9 # test_label_dir = os.path.join(base_data_dir, "test", "labels")
10 # -----
11 # Check if directories exist
12 if os.path.isdir(train_img_dir):
13     print(f"Train image directory: {train_img_dir}")
14 else:
15     print(f"Error: Train image directory not found: {train_img_dir}")
16 if os.path.isdir(train_label_dir):
17     print(f"Train label directory: {train_label_dir}")
18 else:
19     print(f"Error: Train label directory not found: {train_label_dir}")
20 if os.path.isdir(val_img_dir):
21     print(f"Validation image directory: {val_img_dir}")
22 else:
23     print(f"Warning: Validation image directory not found: {val_img_dir}")
24 if os.path.isdir(val_label_dir):
25     print(f"Validation label directory: {val_label_dir}")
26 else:
27     print(f"Warning: Validation label directory not found: {val_label_dir}")
```

```
Train image directory: Areal.v1i.yolov8\train\images
Train label directory: Areal.v1i.yolov8\train\labels
Validation image directory: Areal.v1i.yolov8\valid\images
Validation label directory: Areal.v1i.yolov8\valid\labels
```

```
[3]: 1 # Define image transformations
2 transform = T.Compose([
3     T.Resize((640, 640)), # Typical input size for YOLOv8
4     T.ToTensor(),
5 ])
6
7 # Create Train Dataset and DataLoader
8 train_dataset = YOLOv8Dataset(img_dir=train_img_dir,
9                               label_dir=train_label_dir,
10                               transform=transform)
11 print(len(train_dataset))
```

```
Debug: Found 93 potential image files in Areal.v1i.yolov8\train\images
Debug: Found 93 potential label files in Areal.v1i.yolov8\train\labels
Debug: Matched 93 image-label pairs.
93
```

3. 데이터셋, 데이터로더 클래스 정의

- Training용 커스텀 DataLoader 정의 및 첫번째 미니배치 정보 확인

```
[4]: 1 train_loader = DataLoader(train_dataset,
2                               batch_size=4,      # Reduced batch size for testing
3                               shuffle=True,       # Shuffle training data
4                               num_workers=0,     # Set to 0 for debugging, especially on Windows
5                               collate_fn=yolo_collate_fn, # Use custom collate function
6                               pin_memory=True)    # Optimize data transfer to GPU if available
7
8 print(f"Train dataset loaded successfully. Total samples: {len(train_dataset)}")
9
10 # Test loading a batch from the DataLoader
11 for i, batch_data in enumerate(train_loader):
12     if batch_data is None or batch_data[0] is None:
13         print(f"--- Failed to load Batch {i+1} (collate_fn returned None) ---")
14         continue # Try next batch
15
16     images, labels = batch_data
17     print(f"\n--- Batch {i+1} ---")
18     print(f"Images batch shape: {images.shape}") # Expected: [batch_size, C, H, W]
19     print(f"Labels batch length: {len(labels)}") # Expected: batch_size (list of tensors)
20     for j, label_tensor in enumerate(labels):
21         print(f"  Label tensor {j} shape: {label_tensor.shape}") # Expected: [Num_objects_in_img_j, 5]
22
23         # Print the actual labels (up to 5 for readability)
24         max_labels_to_show = min(5, label_tensor.shape[0])
25         for k in range(max_labels_to_show):
26             cls_id, x, y, w, h = label_tensor[k].tolist()
27             print(f"    Box {k}: class={int(cls_id)}, x={x:.4f}, y={y:.4f}, w={w:.4f}, h={h:.4f}")
28
29         # If there are more labels than shown, indicate this
30         if label_tensor.shape[0] > max_labels_to_show:
31             print(f"    ... and {label_tensor.shape[0] - max_labels_to_show} more boxes")
32     visualize_batch(images, labels)
33     if i == 0:
34         break # Only check the first batch
```

Train dataset loaded successfully. Total samples: 93

3. 데이터셋, 데이터로더 클래스 정의

- Training용 커스텀 DataLoader 정의 및 첫번째 미니배치 정보 확인

```
--- Batch 1 ---  
Images batch shape: torch.Size([4, 3, 640, 640])  
Labels batch length: 4  
Label tensor 0 shape: torch.Size([2, 5])  
Box 0: class=0, x=0.4423, y=0.4928, w=0.6406, h=0.4291  
Box 1: class=0, x=0.5529, y=0.1526, w=0.2380, h=0.1743  
Label tensor 1 shape: torch.Size([2, 5])  
Box 0: class=0, x=0.3654, y=0.8185, w=0.6058, h=0.3558  
Box 1: class=0, x=0.4940, y=0.2825, w=0.5529, h=0.4820  
Label tensor 2 shape: torch.Size([1, 5])  
Box 0: class=0, x=0.4495, y=0.3762, w=0.5493, h=0.4483  
Label tensor 3 shape: torch.Size([1, 5])  
Box 0: class=1, x=0.4627, y=0.5072, w=0.6274, h=0.4423
```



3. 데이터셋, 데이터로더 클래스 정의

- Validation용 커스텀 DataLoader 정의

```
[5]: 1 # Create Validation Dataset and DataLoader
2 # Only create validation loader if directories exist
3 if os.path.isdir(val_img_dir) and os.path.isdir(val_label_dir):
4     val_dataset = YOLOv8Dataset(img_dir=val_img_dir,
5                                 label_dir=val_label_dir,
6                                 transform=transform)
7
8     if len(val_dataset) > 0:
9         val_loader = DataLoader(val_dataset,
10                                batch_size=4,
11                                shuffle=False, # No need to shuffle validation data
12                                num_workers=0,
13                                collate_fn=yolo_collate_fn,
14                                pin_memory=True)
15         print(f"\nValidation dataset loaded successfully. Total samples: {len(val_dataset)}")
16     else:
17         print("Warning: No valid samples found after creating Validation dataset.")
18 else:
19     print("Warning: Validation directories not valid. Skipping Validation DataLoader creation.")
```

```
Debug: Found 25 potential image files in Areal.v1i.yolov8\valid\images
Debug: Found 25 potential label files in Areal.v1i.yolov8\valid\labels
Debug: Matched 25 image-label pairs.
```

```
Validation dataset loaded successfully. Total samples: 25
```

3. 데이터셋, 데이터로더 클래스 정의

- Validation용 커스텀 DataLoader 정의

3. Load Model & Forward pass

```
[6]: 1 import torch
2 import yaml
3
4 # --- 1. Manually Set Command-Line Arguments ---
5 class DummyOpt:
6     pass
7
8 opt = DummyOpt()
9 opt.workers = 8
10 opt.device = "cpu" # Set to "cpu" if GPU is not available
11 opt.batch_size = 4
12 opt.data = "data/coco.yaml" # Data configuration file (includes number of classes, names, etc.)
13 opt.img_size = [640, 640] # Image sizes for [train, test]
14 opt.cfg = "cfg/training/yoloOW.yaml" # Model configuration file
15 opt.weights = "" # If empty, create model from scratch (no pretrained weights)
16 opt.name = "rncs-yolo"
17 opt.hyp = "data/hyp.scratch.p5.yaml" # Hyperparameter file
18 opt.single_cls = False # Whether to treat data as a single class
19
20 # --- 2. Load Device, Hyperparameters, and Dataset Information ---
21 device = torch.device("cpu")
22 print("Device:", device)
23
24 with open(opt.hyp, 'r') as f:
25     hyp = yaml.safe_load(f)
26 print("Hyperparameters loaded from", opt.hyp)
27
28 with open(opt.data, 'r') as f:
29     data_dict = yaml.safe_load(f)
30 nc = 1 if opt.single_cls else int(data_dict['nc'])
31 names = data_dict['names']
32 print(f"Number of classes: {nc}, names: {names}")
```

Device: cpu

Hyperparameters loaded from data/hyp.scratch.p5.yaml

Number of classes: 2, names: ['airplane', 'tennis court']

4. 모델 forward pass 및 loss 계산

- Option 객체를 통해 가져온 모델 모듈 선언
- 선언된 Training용 Loader의 1번 배치 확인
- 레이블 배열에 이미지 index 포함 후 이미지 단위에서 객체 단위로 분할

```
[12]: 1 # --- 3. Load Model (Using Model class from models/yolo.py) ---
2 from models.yolo import Model
3 model = Model(opt.cfg, ch=3, nc=nc).to(device)
4 model.train() # Set model to training mode
5 print("Model loaded from", opt.cfg)
6
7 # Assign additional attributes required by ComputeLoss (e.g., hyp and gr)
8 model.hyp = hyp
9 model.gr = 1.0
10
11 # --- 4. Get a Batch from the DataLoader ---
12 # Assume that a custom DataLoader (train_loader) has already been created.
13 batch = next(iter(train_loader))
14 images, labels = batch
15 print("\nUsing batch from train_loader:")
16 print(f"Images batch shape: {images.shape}")
17 print(f"Number of label tensors: {len(labels)}")
18 for j, label_tensor in enumerate(labels):
19     print(f"Label tensor {j} shape: {label_tensor.shape}")
20     print(label_tensor)
21 visualize_batch(images, labels)
```

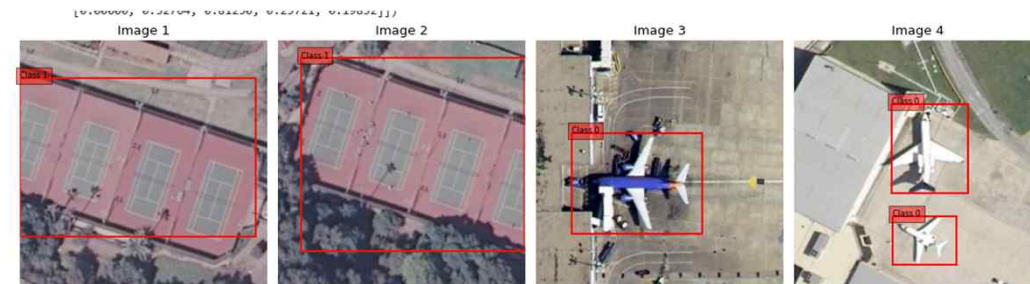
Model loaded from cfg/training/yolo0W.yaml

Using batch from train_loader:

Images batch shape: torch.Size([4, 3, 640, 640])

Number of label tensors: 4

Label tensor 0 shape: torch.Size([1, 5])
tensor([[1.00000, 0.47716, 0.47476, 0.95433, 0.64543]])
Label tensor 1 shape: torch.Size([1, 5])
tensor([[1.00000, 0.54447, 0.46274, 0.90625, 0.78125]])
Label tensor 2 shape: torch.Size([1, 5])
tensor([[0.00000, 0.40986, 0.58173, 0.53245, 0.40865]])
Label tensor 3 shape: torch.Size([2, 5])
tensor([[0.00000, 0.54928, 0.43870, 0.31010, 0.36178],
[0.00000, 0.52764, 0.81250, 0.25721, 0.19832]])



```
1 # --- 5. Construct Targets Tensor ---
2 # For YOLO Loss, each Label needs to include the image index.
3 all_targets = []
4 for i, lab in enumerate(labels):
5     if lab.shape[0] > 0:
6         image_idx = torch.full((lab.shape[0], 1), i, dtype=lab.dtype, device=lab.device)
7         all_targets.append(torch.cat((image_idx, lab), dim=1))
8 if all_targets:
9     targets_tensor = torch.cat(all_targets, dim=0) # Shape: [total_objects, 6]
10 else:
11     targets_tensor = torch.empty((0, 6), device=device)
12 print(f"Constructed targets tensor of shape: {targets_tensor.shape}")
13 print(targets_tensor)
```

Constructed targets tensor of shape: torch.Size([5, 6])

tensor([[0.00000, 1.00000, 0.47716, 0.47476, 0.95433, 0.64543],
[1.00000, 1.00000, 0.54447, 0.46274, 0.90625, 0.78125],
[2.00000, 0.00000, 0.40986, 0.58173, 0.53245, 0.40865],
[3.00000, 0.00000, 0.54928, 0.43870, 0.31010, 0.36178],
[3.00000, 0.00000, 0.52764, 0.81250, 0.25721, 0.19832]])

4. 모델 forward pass 및 loss 계산

- Forward 실행(이미지를 모델에 입력)
- 손실 함수 계산
 - Box Loss
 - Object Loss
 - Classification Loss
 - Total Loss
- Backward 실행(기울기 계산)
 - backward() 이후 파라미터에 grad가 생성됨

```
[9]: 1 # --- 6. Run Model Forward Pass ---
2 outputs = model(images)
3 print("--- Forward Pass ---")
4
5 # --- 7. Compute Loss (Using ComputeLoss) ---
6 from utils.loss import ComputeLoss
7 compute_loss = ComputeLoss(model) # ComputeLoss is defined in utils/loss.py
8 loss, loss_items = compute_loss(outputs, targets_tensor)
9 # Loss items: [box_loss, obj_loss, cls_loss, total_loss]
10 print("\nLoss components:")
11 print(f" Box Loss: {loss_items[0].item()}")
12 print(f" Obj Loss: {loss_items[1].item()}")
13 print(f" Cls Loss: {loss_items[2].item()}")
14 print(f" Total Loss: {loss_items[3].item()}")
15
16 print("Before backward() call. Gradient values:")
17 for name, param in model.named_parameters():
18     if param.grad is not None:
19         grad_norm = param.grad.norm().item()
20         print(f" {name} gradient norm: {grad_norm}")
21         break
22 loss.backward()
23 print("\nAfter backward() call. Gradient values:")
24 for name, param in model.named_parameters():
25     if param.grad is not None:
26         grad_norm = param.grad.norm().item()
27         print(f" {name} gradient norm: {grad_norm}")
28         break
29
30 print("\nForward and backward pass with ComputeLoss completed successfully.")

--- Forward Pass ---

Loss components:
Box Loss: 0.04549568146467209
Obj Loss: 0.01689322479069233
Cls Loss: 0.3636641800403595
Total Loss: 0.42605307698249817
Before backward() call. Gradient values:
After backward() call. Gradient values:
model.0.conv1.rbr_dense.0.weight gradient norm: 56.48033142089844
Forward and backward pass with ComputeLoss completed successfully.
```

Thank you